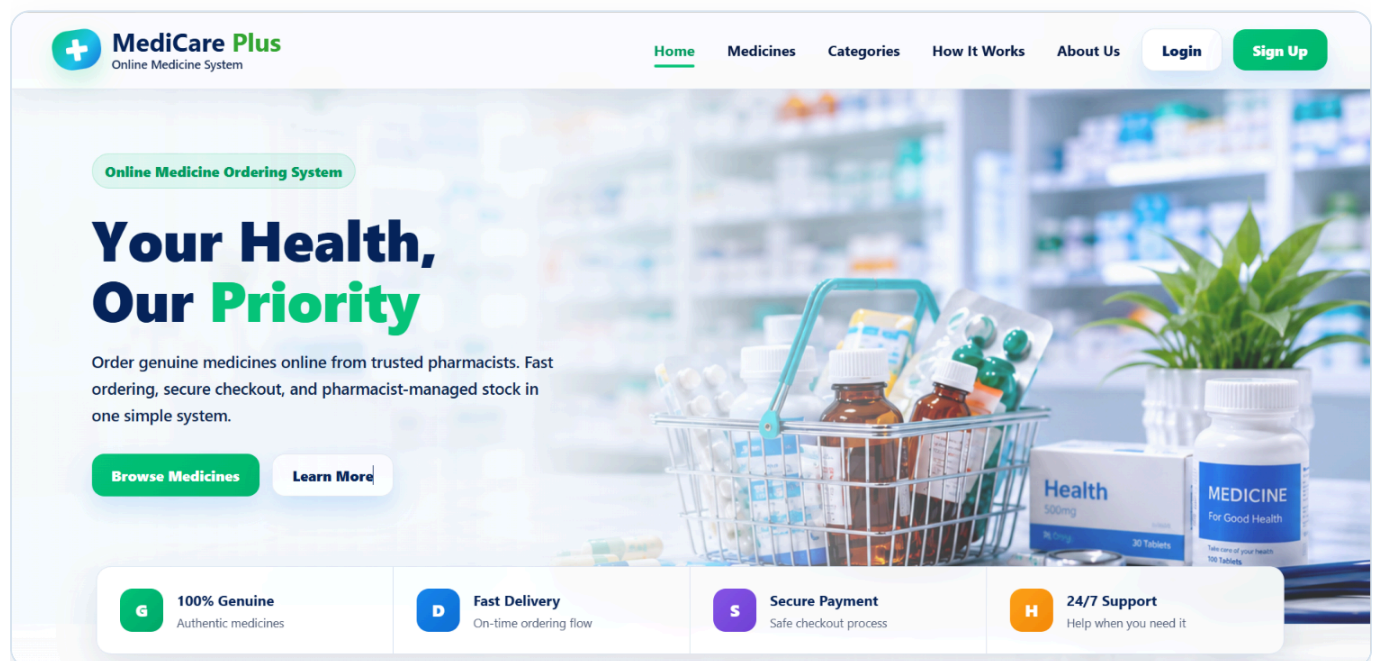


PROJECT REPORT

MediCare Plus Online Medicine Management System

A Java JSP and Servlet based online pharmacy management system developed as our Second Year, First Semester OOP module project.



Project Type

Java JSP Servlet MVC-style web application

Database

MySQL with JDBC

Server

Apache Tomcat 10.x

Build

Maven WAR project

1. Project Overview

MediCare Plus was developed to demonstrate how an online medicine ordering platform can be designed using Java web technologies and object-oriented development ideas. Our project connects three main user groups: customers, pharmacists, and admins.

The customer side focuses on finding medicines, managing a cart, selecting delivery methods, placing orders, tracking status, and contacting admin support. The pharmacist side focuses on inventory management, stock ownership, order processing, delivery ETA handling, and analytics. The admin side controls pharmacist approvals, account management, order monitoring, and customer support messages.

Academic context: This project was completed for a Second Year, First Semester OOP module. We used Java classes, DAO objects, servlet controllers, JSP views, and a MySQL schema to keep the design understandable and maintainable.

2. Problem and Objectives

Traditional medicine ordering often depends on manual phone calls or direct visits. Stock availability, delivery status, and pharmacist communication can become unclear. MediCare Plus addresses that problem by creating one shared web application where each role has a focused dashboard and clear responsibilities.

Main objectives

- Allow customers to browse available pharmacist stock and place medicine orders online.
- Allow pharmacists to manage medicine stock, images, transport methods, and fulfilment status.
- Allow admins to approve pharmacists, monitor platform activity, and respond to customer issues.
- Apply OOP and MVC-style separation through models, DAOs, servlets, and JSP pages.
- Use a MySQL database with clean relational tables for users, medicines, carts, orders, order items, and support messages.

3. Technology Stack

Layer	Technology	Use in our project
Language	Java 11	Models, DAO logic, servlets, validation, and database operations.
Web views	JSP	Role-based pages for public, customer, pharmacist, and admin areas.
Controller	Jakarta Servlets	Form handling, session flow, redirects, and role actions.
Database access	JDBC	Direct SQL operations through DAO classes.
Database	MySQL	Stores all accounts, stock, carts, orders, order items, and messages.
Build	Maven	Dependency management and WAR artifact configuration.
Server	Tomcat 10.x	Runs the Jakarta Servlet based web application.
Frontend	HTML, CSS, JavaScript	Responsive pages, dashboards, live cart total, filters, and UI feedback.

4. Architecture

We structured the project as a Maven WAR web application. The design follows an MVC-style separation where JSP files handle views, servlets handle requests, DAO classes handle data operations, and model classes represent application objects.

Part	Folder	Responsibility
Model	src/main/java/com/medicare/model	Represents data objects such as users and medicines.
DAO	src/main/java/com/medicare/dao	Contains database access and key business operations.
Controller	src/main/java/com/medicare/servlet	Receives user actions and redirects users after processing.
View	src/main/webapp	JSP pages for public, admin, pharmacist, and customer interfaces.
Assets	src/main/webapp/assets	CSS, JavaScript, and image files.
Database	database/online_medicine_db.sql	Creates tables and seed data.

This structure makes the repository easy to review. For example, the cart and order rules are mainly in `CartOrderDAO`, while the pharmacist medicine operations are in `MedicineDAO`. That separation keeps JSP pages from becoming responsible for database-heavy logic.

5. Database Design

The database is named `online_medicine_db`. It contains six main tables.

Table	Main data	Why it is needed
users	Account details, role, pharmacy details, approval status.	Supports admin, pharmacist, and customer accounts in one table.
medicines	Name, category, price, quantity, expiry, image, delivery methods, owner pharmacist.	Stores stock that customers browse and pharmacists manage.
cart	Customer, medicine, quantity.	Stores selected items before checkout.
orders	Customer, total, status, delivery method, date.	Stores the main order record.
order_items	Order medicines, item status, expected delivery time.	Allows pharmacist-level fulfilment and item-level status tracking.
order_issues	Customer message, issue type, admin response, status.	Supports after-order communication and admin support.

6. Main Workflows

Customer ordering workflow

1. The customer logs in and opens the medicine list.
2. The system shows only medicines from approved pharmacists with available quantity.
3. The customer selects quantity and adds items to the cart.
4. The cart calculates row subtotal and grand total.
5. The customer selects a delivery method shared by the medicines in the cart.
6. The order is placed inside a database transaction.
7. Stock is reduced only after the order is created successfully.
8. The customer tracks the order by status and can message admin if there is an issue.

Pharmacist fulfilment workflow

1. The pharmacist manages medicine stock and delivery options.
2. The pharmacist sees only orders containing medicines created by that pharmacist.
3. The pharmacist reviews the order and updates fulfilment status.
4. When status is Approved, the pharmacist must enter expected delivery date/time.
5. The customer sees Approved orders as On The Way with ETA.
6. The pharmacist can complete or cancel the order later.

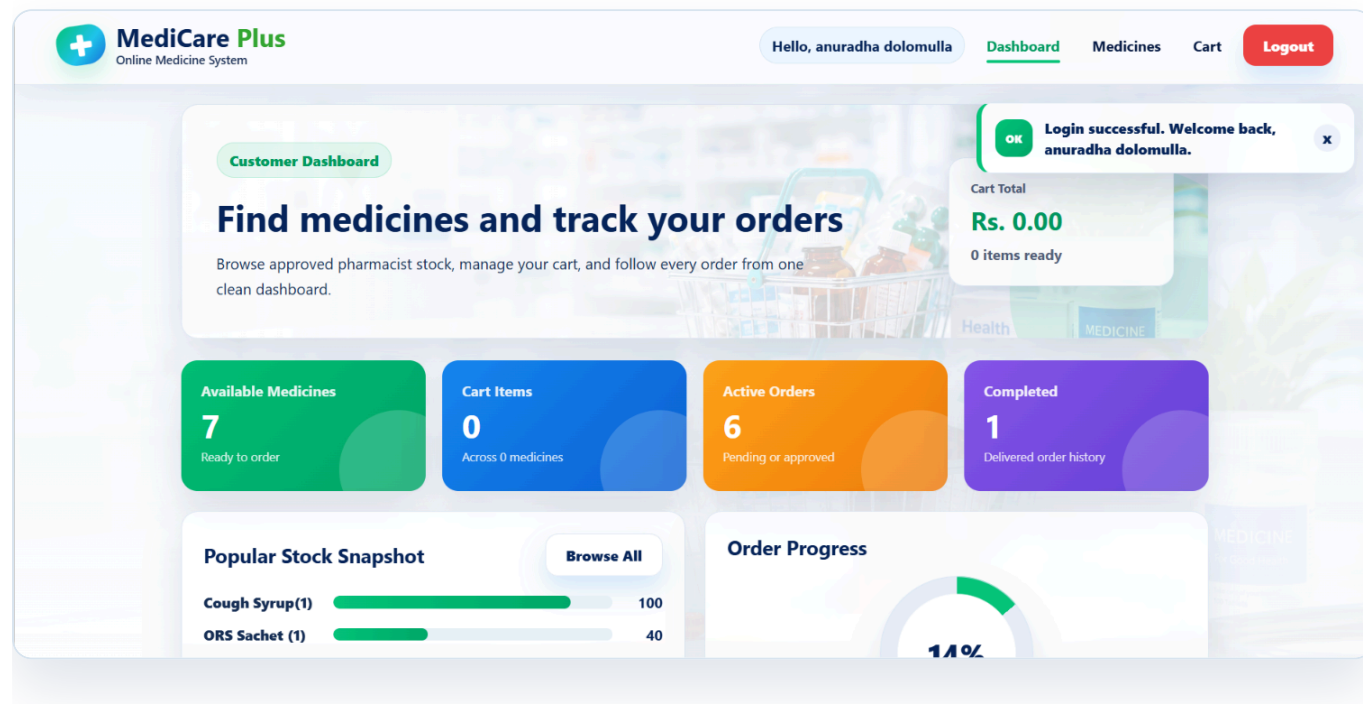
Admin support workflow

1. Admin reviews pending pharmacist accounts and approves valid accounts.
2. Admin monitors users, pharmacists, orders, and platform activity.
3. Customers send order issues from the order page.
4. Admin replies and updates the message status.
5. The customer's order page shows admin responses under the relevant order.

7. Role-Based Feature Details

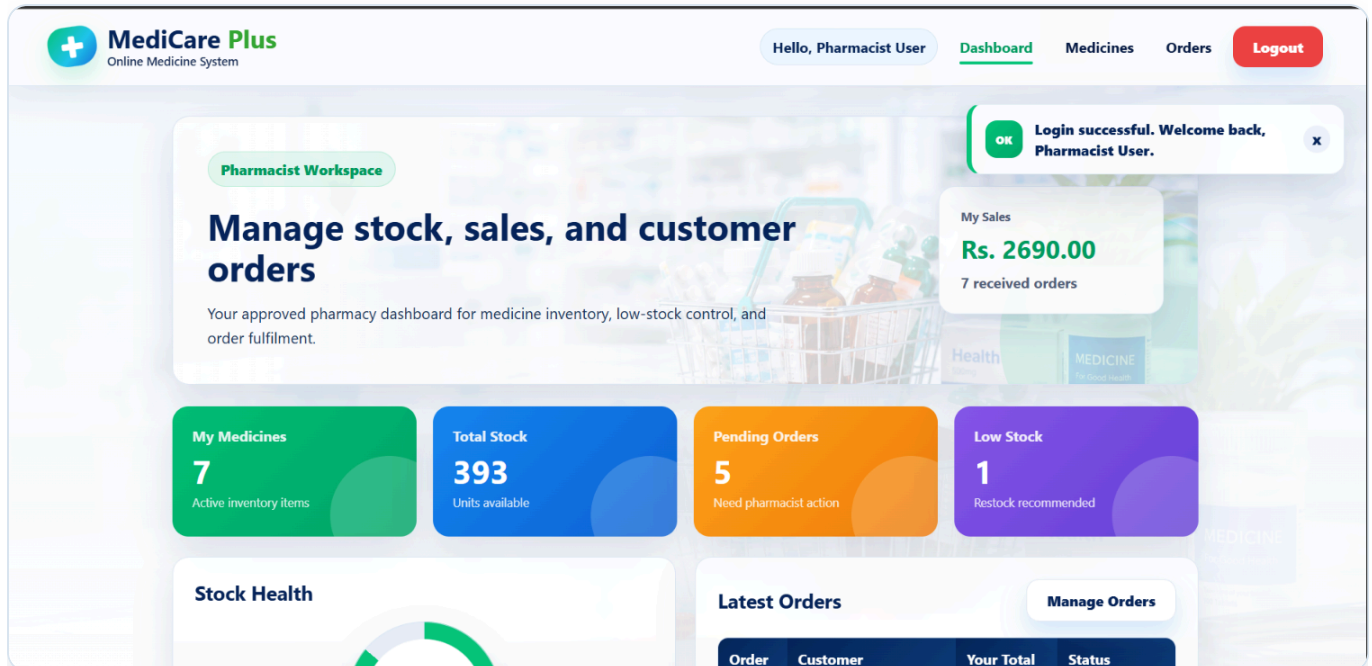
Customer module

The customer module is one of the most advanced parts of the system because it covers a complete order journey. The user can browse approved stock, add items to cart, select a valid transport method, place an order, track order progress, and communicate with admin support. The cart and order logic also includes stock checks and delivery method validation, so invalid orders are stopped before they reach the database.



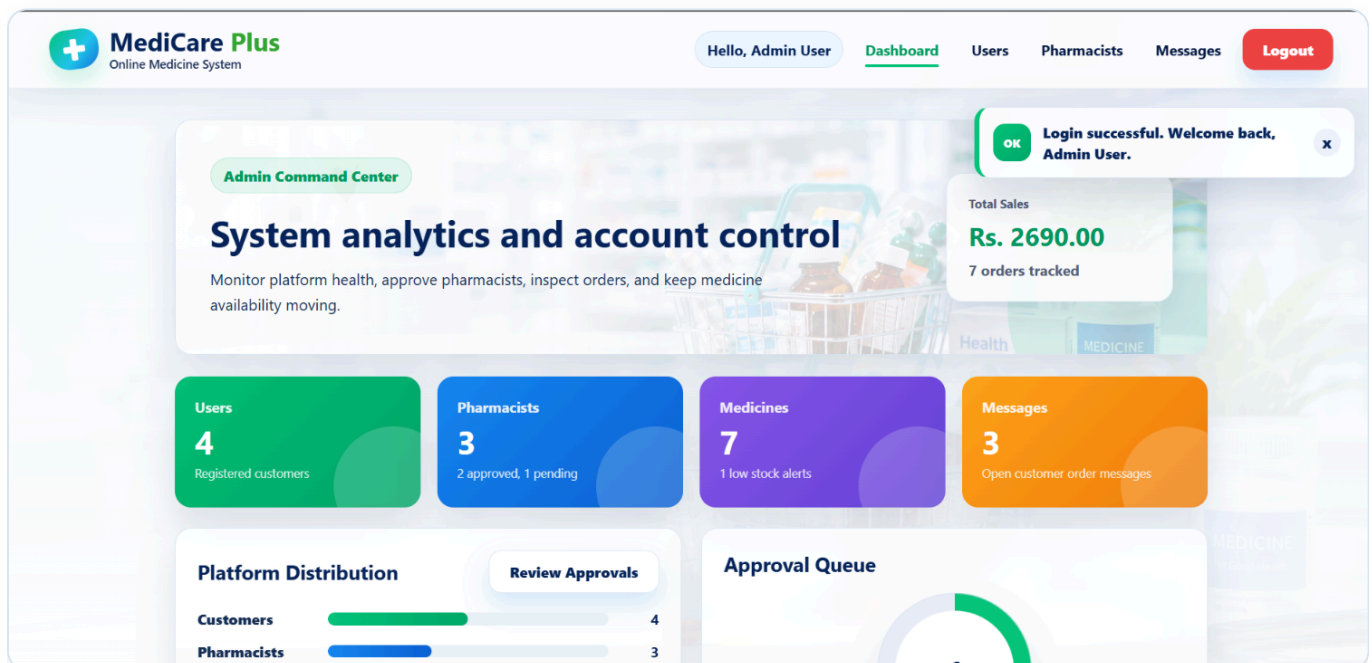
Pharmacist module

The pharmacist module is the second major advanced area. It covers inventory ownership, medicine image upload, delivery service configuration, order fulfilment, expected delivery handling, and analytics. The pharmacist can only edit their own medicines and only sees orders connected to their own stock.



Admin module

The admin module controls platform safety and support. Admins approve pharmacists, manage users and pharmacists, monitor orders, and respond to customer messages. This keeps the system controlled while still allowing pharmacists and customers to use their own workflows.



Authentication module

The authentication module provides the base for the rest of the application. It handles account registration, login, logout, password hashing, role-based redirects, and pending pharmacist

blocking. Without this foundation, the customer, pharmacist, and admin modules would not be separated safely.

8. Team Contribution Summary

Member	Branch	Contribution
Anuradha	feature/anuradha-customer-ordering-delivery	Customer medicine browsing, cart, live totals, delivery selection, order placement, order tracking, issue sending, and customer-facing order status views.
Damsi	feature/damsi-pharmacist-inventory-analytics	Pharmacist dashboard, medicine CRUD, image upload, stock control, delivery methods, order fulfilment, ETA handling, low stock, and analytics.
Supuni	feature/supuni-auth-role-access	Login, registration, password hashing, sessions, logout, role-based redirects, and pending pharmacist blocking.
Pabasha	feature/pabasha-admin-approval-support	Admin dashboard, user and pharmacist management, pharmacist approval, order monitoring, support messages, admin responses, and message statuses.

Advanced feature areas: The most workflow-heavy areas are Anuradha's customer ordering and delivery module, followed by Damsi's pharmacist inventory and fulfilment module. Pabasha's admin support module adds management depth, while Supuni's authentication module provides the required access-control foundation.

9. Setup Guide

1. Install JDK 11 or higher.
2. Install Maven.
3. Install MySQL Server or XAMPP MySQL.
4. Install Apache Tomcat 10.x.
5. Open the project as a Maven project in IntelliJ IDEA, Eclipse Enterprise, or NetBeans.
6. Import `database/online_medicine_db.sql` into MySQL.
7. Update `src/main/java/com/medicare/config/DBConnection.java` with the local MySQL username and password.
8. Deploy the WAR exploded artifact with context path `/online_medicine_system`.
9. Open `http://localhost:8080/online_medicine_system/`.

Sample accounts

Role	Email	Password
Admin	admin@gmail.com	123456
Pharmacist	pharmacist@gmail.com	123456
Customer	user@gmail.com	123456

10. Conclusion

MediCare Plus shows how an OOP module web project can be extended into a complete role-based medicine ordering system. The project includes a practical customer flow, pharmacist inventory and fulfilment logic, admin approval and support features, and authentication control. We used Java, JSP, Servlets, JDBC, MySQL, and a clean folder structure to keep the application understandable for future review and improvement.

Generated from the repository documentation and UI screenshots captured during development.